

Prisma

Schema-First ORM for TypeScript

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand why an ORM beats raw SQL strings
- ▶ Define a database schema in Prisma and run migrations
- ▶ Perform CRUD with the Prisma Client
- ▶ Model one-to-many relations with `include`
- ▶ Swap from SQLite to Postgres with one line

Section Items

- ▶ Why an ORM?
- ▶ Schema and Migrations
- ▶ Prisma Client CRUD
- ▶ Relations
- ▶ Postgres for Production

Section Items

► Why an ORM?

- Schema and Migrations
- Prisma Client CRUD
- Relations
- Postgres for Production

Three Options for Talking to a Database

- ▶ **Raw SQL strings** (pg/mysql2) — works, tedious, easy to forget parametrization
→ SQL injection
- ▶ **Query builder** (Knex) — better than strings, still verbose
- ▶ **ORM** (Prisma) — schema-first, types generated, parametrized by construction

The Node ORM Landscape

	Prisma	Drizzle	TypeORM
Style	Schema-first DSL	SQL-like TS API	Decorators on classes
Type safety	Excellent (codegen)	Excellent	Mediocre
Migrations	Yes, owned tool	Yes	Yes (less ergonomic)
Community	Huge	Growing fast	Shrinking

We teach Prisma: smallest “new things to learn” surface for students who already know TypeScript.

Section Items

- Why an ORM?

► **Schema and Migrations**

- Prisma Client CRUD
- Relations
- Postgres for Production

The Prisma Schema

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "sqlite"  
  url      = env("DATABASE_URL")  
}  
  
model Todo {  
  id          Int          @id @default(autoincrement())  
  text        String  
  done        Boolean      @default(false)  
  minutes     Int  
  createdAt   DateTime     @default(now())  
}
```


What prisma migrate dev Does

```
npx prisma migrate dev --name init
```

1. Diffs the schema against the database
2. Generates a SQL migration file
3. Applies it
4. Regenerates the TypeScript Prisma Client

Inspecting Migrations

- ▶ `prisma/migrations/*/migration.sql` — the raw SQL
- ▶ Reviewable, versioned, diff-able in PRs
- ▶ `npm run db:studio` — a tabbed GUI for browsing and editing rows

- ▶ Open `prisma/schema.prisma` — the single source of truth
- ▶ Run `npx prisma migrate dev --name init`
- ▶ Open `prisma/migrations/` — see the generated SQL
- ▶ Run `npx prisma studio` — browse the empty table

Section Items

- Why an ORM?
- Schema and Migrations
- ▶ **Prisma Client CRUD**
 - Relations
 - Postgres for Production

```
prisma.todo.findMany({ where: { done: false } });
prisma.todo.findUnique({ where: { id: 1 } });
prisma.todo.create({ data: { text: "hi", minutes: 15 } });
prisma.todo.update({ where: { id: 1 }, data: { done: true } });
prisma.todo.delete({ where: { id: 1 } });
prisma.todo.count();
```

The Before / After

Express demo (in-memory):

```
const todo = todoStore.create(text, minutes);
```

This snapshot (database):

```
const todo = await prisma.todo.create({  
  data: { text, minutes },  
});
```

Same logic. Different layer. Same TypeScript types.

Async Glue: `asyncHandler`

```
asyncHandler(async (req, res) => {  
  const todos = await prisma.todo.findMany();  
  res.json(todos);  
});
```

- ▶ Express 4 doesn't catch thrown errors from async functions
- ▶ Wrap each handler with `asyncHandler`
- ▶ Express 5 doesn't need this — we're on 4 for now

Pre-check for 404

```
const existing = await prisma.todo.findUnique({
  where: { id },
});
if (!existing) {
  res.status(404).json({ error: "Not found" });
  return;
}
await prisma.todo.update({ where: { id }, data: ... });
```

Alternative: let Prisma throw P2025 and catch it. Both fine. Pre-check is more obvious for beginners.

Live Code: Prisma CRUD

- ▶ Add, toggle, delete todos through the frontend
- ▶ Open Prisma Studio — see the rows in the database
- ▶ Restart the server — data persists (it's in SQLite now)

Section Items

- Why an ORM?
- Schema and Migrations
- Prisma Client CRUD
- ▶ **Relations**
 - Postgres for Production

One-to-Many in Prisma

```
model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique
  todos   Todo[]           // back-relation
}

model Todo {
  id      Int      @id @default(autoincrement())
  text    String
  category Category? @relation(fields: [categoryId],
                                references: [id])
  categoryId Int?      // the actual FK column
}
```

include = JOIN

```
prisma.todo.findUnique({
  where: { id },
  include: { category: true },
});
// -> { id, text, ..., category: { id, name } }

prisma.category.findUnique({
  where: { id },
  include: { todos: true },
});
// -> { id, name, todos: [{...}, {...}] }
```

Only include what you'll use — each adds a query.

Error Codes Worth Catching

Code	Meaning	HTTP
P2002	Unique constraint failed	409
P2003	FK constraint failed	400
P2025	Record not found	404

Catching Prisma Errors

```
try {  
  await prisma.category.create({ data: { name } });  
} catch (err) {  
  if (err.code === "P2002")  
    return res.status(409).json({ error: "name taken" });  
  throw err;  
}
```

onDelete Policy

```
category Category? @relation(  
  fields: [categoryId], references: [id],  
  onDelete: SetNull  
)
```

- ▶ Cascade — delete the dependents
- ▶ SetNull — null the FK on the dependents
- ▶ Restrict (default) — block the delete

- ▶ Open Prisma Studio — see Category and Todo tables
- ▶ Create a category, assign todos to it
- ▶ Delete the category — watch todos become uncategorized (SetNull)

Section Items

- Why an ORM?
- Schema and Migrations
- Prisma Client CRUD
- Relations
- ▶ **Postgres for Production**

The One-Line Swap

```
datasource db {  
-   provider = "sqlite"  
+   provider = "postgresql"  
    url      = env("DATABASE_URL")  
}
```

That's it. Routes, middleware, seed — unchanged.

Why Swap from SQLite?

SQLite is a single file. Production needs:

- ▶ Many connections at once (web app + cron + worker + dashboard)
- ▶ Network access (your app on Render can't read a SQLite file on Vercel)
- ▶ Backups, replication, observability

Neon — Serverless Postgres

- ▶ Free tier sufficient for class projects
- ▶ Browser dashboard — copy connection string, paste into DATABASE_URL
- ▶ Branched databases (`main`, `pr-42`) — useful for final project teams

Deploy Commands (Overview)

Locally, against Neon:

```
npx prisma migrate dev --name init
npx prisma db seed
```

On Render, in the build command:

```
npm install
npx prisma migrate deploy      # applies pending, never resets
npx prisma generate            # regenerate Client
```

migrate dev = local (may reset). migrate deploy = production (safe).

What You Can Do Now

- ▶ Define a database schema in `schema.prisma` ✓
- ▶ Run migrations with `prisma migrate dev` ✓
- ▶ Perform typed CRUD with Prisma Client ✓
- ▶ Model relations and use `include` ✓
- ▶ Swap SQLite → Postgres with one line ✓

The Mental Model

- ▶ Prisma = schema in `schema.prisma`, types in your IDE, SQL on disk, runtime Client
 - ▶ `migrate dev` for local; `migrate deploy` for production
 - ▶ `include` to hydrate relations
 - ▶ Error codes (P2002, P2003, P2025) → clean HTTP statuses
 - ▶ **Engine swap is one line** — build on SQLite, deploy to Postgres
- Next:** wiring this into the React + TanStack frontend, then deploy (Vercel + Render + Neon).